README.md

NGP (Native Gadget Programming)

NGP and the NGP compiler is now available (`ngp.py`)

The NGP compiler has successfully mapped the `msfvenom` payload and bypassed antivirus detection ~~(EDR **not tested**)~~.

The result of EDR evasion can be found in the **NGP Dropper Test** section.

The NGP compiler only provides metadata and is intentionally designed not to include a dropper that reconstructs and loads the actual shellcode.

~~NGP and the NGP compiler are currently under development, and **this document addresses theoretical concepts only.**~~

Created by @therustymate

Youtube Video: https://youtu.be/1r0l6spXKCI

Disclaimer

This document and all associated materials are provided strictly for **legitimate security research, education, and authorized antivirus detection capability testing purposes only.** The techniques and concepts described herein involve advanced software security, malware analysis, and development methods, and **any unauthorized use, reproduction, distribution, or malicious deployment against systems without explicit permission is strictly prohibited.**

By accessing and utilizing this material, you acknowledge and agree to comply with all applicable laws and regulations, and to obtain proper authorization before conducting any security testing or research activities.

The author and affiliated parties **expressly disclaim all legal liability and responsibility for any misuse, unauthorized actions, or damages arising from the use of this information.**

Furthermore, this research was conducted to study current antivirus detection limitations, develop evasion techniques for educational purposes, and enhance cybersecurity expertise. The disclosure of this technology is purely for advancing the security industry and academic research.

Therefore, all risks, legal responsibilities, and consequences resulting from the use or misuse of this document rest solely with the user. The author and related parties are fully indemnified from any direct or indirect damages.

By reading or using this document, you are deemed to have accepted all the above conditions.

NGP Operation Principle

NGP, or **Native Gadget Programming**, is a novel technique designed to execute malicious code without embedding the actual shellcode within the shellcode dropper. This technology leverages byte fragments from binaries already present on the system to **reconstruct and execute shellcode in memory**, effectively evading antivirus detection.

Operation Process

1. **Shellcode Analysis and Mapping -** The NGP compiler analyzes the provided shellcode and searches for **matching or similar byte sequences** within default Windows system files (e.g., executables, DLLs, other binary files).
2. **Fragment Location Storage -** When matching byte fragments are found, metadata such as **file path**, **file offset**, and **length** are recorded and hardcoded into the final dropper binary.
3. **Dynamic Building at Runtime -** When the dropper executes, it opens the target files, reads the necessary byte fragments, and sequentially **rebuilds the shellcode in memory**.
4. **Shellcode Execution -** Finally, the reconstructed shellcode is loaded and executed in memory using APIs like `VirtualAlloc` , `memcpy`

Because the dropper does not contain fully executable shellcode but only incomplete fragment information, this approach is highly effective at evading signature-based antivirus detection.

Users can also partially evade detection of complete malicious binaries performing harmful actions via NGP. The NGP compiler analyzes the provided binary, maps portions to byte fragments within legitimate system files. At runtime, these fragments are reassembled in memory and loaded directly via techniques like shellcode reflective loading, effectively bypassing static antivirus analysis.

Signature-Based Detection Evasion Mechanism

Signature-based detection identifies malware by matching unique byte patterns or code fragments stored in a database against files. This method achieves high detection rates when explicit malicious code fragments exist within a file.

However, NGP effectively evades signature detection for the following reasons:

- **Absence of Complete Malicious Code Patterns –** The dropper binary does not contain executable malicious bytes internally; instead, malicious code is fetched piecewise from external legitimate system files and reconstructed in memory, so no full malicious signature exists within the file.
- **Distributed and Reused Code Fragments –** Malicious code is fragmented and distributed in memory; these fragments exist identically or similarly within default Windows system files, reducing uniqueness and making detection difficult.
- **Use of Legitimate System File Code –** By reusing code fragments from legitimate files, antivirus products hesitate to classify these as malicious, lowering detection likelihood.

Consequently, NGP malware's complete execution sequence is not present as a whole within a single file but dispersed and reassembled, making it difficult for traditional signature-based detection methods to effectively detect.

YARA Rule Detection and NGP Evasion Potential

YARA is a tool that identifies malware based on specific patterns, strings, binary sequences, or regular expressions, widely used in static analysis and memory scanning. YARA rules typically include unique byte sequences, function signatures, and strings found in malware samples.

However, NGP is likely to evade YARA detection due to:

- **Distributed and Reassembled Malware Structure –** NGP splits malware into fragments that reference system binaries, reassembled only at runtime, meaning unique continuous byte patterns or strings do not exist within a single executable. This complicates YARA's pattern matching.

- **Use of Legitimate System File Code –** Since malware fragments match bytes inside legitimate system files, writing YARA rules based on these fragments risks false positives and limits rule application.

Thus, NGP-based malware exhibits a high evasion rate against traditional YARA rules, requiring dynamic analysis or memory behavior-based detection techniques for effective identification.

Major Differences and Implications Between Encrypted Shellcode and NGP

1. **Difficulty of Automated Full Content Analysis and Detection**

- NGP fragments malicious code across many legitimate system files, forcing antivirus to access and reconstruct fragments from numerous files, a complex and resource-intensive task.
- This leads to overall system performance degradation during detection, undesirable for users or administrators.
- For attackers, increased detection cost and resource consumption degrade defenders' detection and response capabilities, enhancing attack success.

2. **Enhanced Static Analysis and Signature Detection Evasion**

- Encrypted shellcode is hard to detect before decryption, but NGP lacks a complete malicious signature in any single executable file, virtually neutralizing signature-based detection.
- Fragmented malware spread over multiple legitimate files requires full assembly for detection, practically impossible without full context.

Limitations and Constraints of NGP Technology

NGP offers strengths in evading static and signature-based detection but faces the following challenges:

1. **Difficulty of Complete Code Mapping**

- Perfectly mapping all byte fragments inside legitimate Windows system files is practically impossible.
- Some shellcode or malware bytes are absent or hard to find in system files, requiring encryption or separate storage.

- This results in encrypted code sections inside the dropper, which may be subject to static or behavioral analysis.

2. **Limitations in Evading Memory-Based Detection (EDR)**

- Runtime behaviors such as API calls, RWX (Read/Write/Execute) memory allocations, and code injections are monitored by modern EDR solutions.
- System calls like `VirtualAlloc`, `WriteProcessMemory`, `CreateRemoteThread`, and `NtResumeThread` are vulnerable to behavioral detection.
- While NGP can hide some activities using ROP techniques, (**EDR-evasive NGP is still under development**), full evasion of dynamic detection remains difficult.

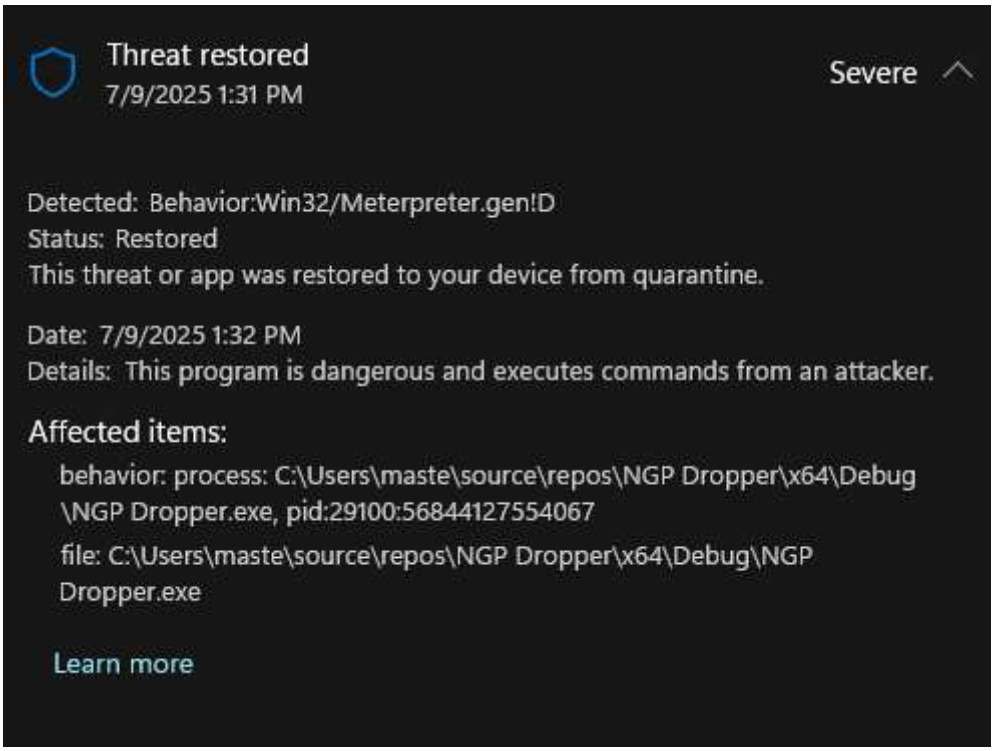
3. **Instability Due to Legitimate File Changes**

- Updates or patches to system files change fragment offsets, causing assembly failures or malfunctions.
- NGP is therefore version-dependent, complicating maintenance and response.

4. **Complex Development and Testing Environment**

- Implementing NGP compilers and droppers requires complex integration of fragment mapping, encryption, decryption, and memory permission adjustments.
- Improper implementation can degrade system stability and cause unexpected errors.

NGP Dropper Test



Static detection by antivirus was completely bypassed, but behavior-based detection failed to be evaded due to the use of the well-known **Meterpreter** payload.